



Experiment No. 13: Structures in C

Objectives

The objectives of this lab are:

1. Understand the concept and need for structures, unions, and enumerations in C programming.
2. Learn to declare, define, and access structures and unions.
3. Differentiate between structures and unions in terms of memory allocation and usage.
4. Understand the use of enumeration types for better code readability and maintainability.
5. Perform bit-level operations including AND, OR, XOR, NOT, shift left, and shift right.
6. Learn bit field declarations and how they can optimize memory usage.
7. Apply the concepts through practical examples and lab exercises.

Prerequisites

Following are the prerequisites of this lab are:

1. Understanding of data types in C programming language.
2. Basic knowledge of variables, loops, functions, arrays, and pointers in C.
3. Understanding of memory allocation in C.
4. Basic knowledge of memory addressing and sorting of arrays.
5. Basic knowledge of binary numbers for bit-level operations.
6. Read chapter 10 from textbook (C-How to Program, 9 Ed.)

Introduction

Structure

Structures (also called structs) are a way to group several related variables into one place. Structures are customized define data types – derived from primitive data types (e.g., char, int, and double, etc.).

Declaration of Structures in C

Structure can be created by using the struct keyword and declare each of its members inside curly braces:

```
struct structName {    // structName is a tag. "STRUCTURE DECLARATION"
    DataType Variable_1;    // Member
    DataType Variable_2;    // Member
    . . .
    DataType Variable_n;    // Member
}; // End the structure with a semicolon
```



Programming Fundamentals Lab Manual



Note: Members must have unique names within a structure, but different structures can reuse member names. Members can be constant and non-constant primitive types (e.g., int, double), arrays, or other structures.

Self-Referential Structures

Self Referential structures are those structures that have one or more pointers which point to the same type of structure, as their member.

```
struct structName {           // structName is a tag (Self-Referential
    DataType member1;         // Member
    DataType member2;         // Member
    struct structName *pointer; // Referencing Pointer to same
```

Note: A structure may not contain a variable of its own type, but it can contain a pointer to the same structure type.

Structure Definition vs Structure Variables

A structure definition in C does not reserves any memory. It only tells the compiler what data members the structure will have, so it acts like a blueprint. Memory is allocated only when variables of the structure type are declared.

```
struct structName {
    DataType Variable_1;
    DataType Variable_2;
};
struct structName myStructure;
struct structName Name[52];
struct structName *pointer;
```

→ Just Create a New Data Type

→ Here, Memory is Allocated

Note: Structure variables can also be declared at the end of the structure definition using a comma-separated list, as shown below.

```
struct structName {
    DataType member1;
    DataType member2;
}myStructure, Name[52], *pointer; //Defining Structures & Variables
simultaneously
```

Note: The structure tag name acts as the name of the structure data type. If a tag name is not used,



Programming Fundamentals Lab Manual

structure variables must be declared immediately, and new variables cannot be created later.

Case 1: Using a structure tag name (recommended)

```
struct structName {           // structName is the tag (data type
name)
    DataType member1;         // Member
    DataType member2;         // Member
    ...
    DataType memberN;
};
struct structName myStructure; // Memory allocated
struct structName array[52];   // Memory allocated for 52 structures
struct structName *pointer;    // Pointer to a structure
```

Case 2: Without using a tag name (tag name optional)

```
struct {                       // No tag name
    DataType member1;         // Member
    DataType member2;         // Member
    ...
    DataType memberN;
};
```

Allowed Structure Operations

The following operations can be performed on structure variables:

- Assignment of one structure variable to another of the same type; for pointer members, only the stored addresses are copied
- Obtaining the address of a structure variable using the address-of operator (&)
- Accessing structure members using the structure member operator (.) and the structure pointer operator (->)
- Determining the size of a structure variable using the sizeof operator
- Zero initialization of a structure variable at the time of definition

Example

```
#include <stdio.h>
struct card /* Structure definition */
{
    int number;
    char suit;
};
int main(void)
{
    struct card c1 = {0}; /* Zero initialization */
```



Programming Fundamentals Lab Manual



```
/* Accessing members using dot operator */
c1.number = 10;
c1.suit = 'H';
printf("c1: Number = %d, Suit = %c\n", c1.number, c1.suit);

/* Assigning one structure to another of the same type */
struct card c2;
c2 = c1;
printf("c2 (after assignment): Number = %d, Suit = %c\n", c2.number,
c2.suit);

/* Taking address of a structure variable */
printf("Address of c1 = %p\n", (void *)&c1);

/* Using sizeof with structure */
printf("Size of struct card = %zu bytes\n", sizeof(struct card));

return 0;
}
```

Output

```
c1: Number = 10, Suit = H
c2 (after assignment): Number = 10, Suit = H
Address of c1 = 000000CB275FF7D8
Size of struct card = 8 bytes
```

Note: Structures cannot be compared using == or != operators because members may not be stored in consecutive memory locations. Memory alignment may introduce padding bytes (“holes”) between members to satisfy machine-specific alignment requirements. Even if all member values of two structures are equal, the padding bytes may differ, making direct comparison unreliable. Memory layout and padding are machine-dependent, so structure equality must be checked member by member.

Structure Initialization

Like arrays, you can initialize a struct variable via an initializer list. A structure can be initialized by providing values in a list during declaration. The values are assigned to the structure members sequentially, with the first value going to the first member, and so on.

```
// Sequential initialization
struct structName var1 = {value1, value2, value3, pointerValue};
// Fewer initializers than members
struct structName var2 = {value1, value2}; // member3 = 0, pointerMember =
NULL automatically
```



Programming Fundamentals Lab Manual



To initialize members non-sequentially, designated initialization can be used, specifying the member names explicitly.

```
struct structName {           // Structure definition
    DataType member1;
    DataType member2;
    DataType member3;
    DataType *pointerMember;   // Pointer member
};

// Designated initialization (non-sequential)
struct structName var3 = {
    .member3 = value3,
    .member1 = value1,
    .pointerMember = pointerValue
    // member2 not initialized → automatically 0
};
```

Note: If a structure has fewer initializers than members, the remaining members are automatically set to 0 (for numeric types) or NULL (for pointers).

Example

```
#include <stdio.h>
struct student {           // Structure definition
    int id;
    float marks;
    char grade;
    int *ptr;               // Pointer member
};

// Sequential initialization
struct student s1 = {101, 85.5, 'A', NULL};
// Fewer initializers than members
struct student s2 = {102, 90.0}; // grade = '\0', ptr = NULL
                                // automatically

int main() {
    // Accessing members to verify
    printf("s1: %d, %.2f, %c, %p\n", s1.id, s1.marks, s1.grade, s1.ptr);
    printf("s2: %d, %.2f, %c, %p\n", s2.id, s2.marks, s2.grade, s2.ptr);

    return 0;
}
```



Programming Fundamentals Lab Manual



Output

```
s1: 101, 85.50, A, 00000000000000000000
s2: 102, 90.00, , 00000000000000000000
```

Accessing Structure Members

Structure members can be accessed in two ways:

1. The structure member operator (.), or dot operator – It is used with structure variables.
2. The structure pointer operator (->), or arrow operator – It is used with a pointer to a structure.

Syntax for accessing structure members is as follow;

```
struct structName var1;

// Declaring a pointer to the structure
struct structName *ptr;

// Assigning values using dot operator (structure variable)
var1.member1 = value1;
var1.member2 = value2;
var1.member3 = value3;
var1.pointerMember = pointerValue;

ptr = &var1;
// Accessing members using arrow operator (pointer)
ptr->member1 = value1;
ptr->member2 = value2;
ptr->member3 = value3;
ptr->pointerMember = pointerValue;
```

Note: Do not put spaces around “.” or “->”. Adding spaces (e.g., ptr - > member) is a syntax error. These operators are treated as part of a single variable access.

Example

```
#include <stdio.h>
// card structure definition
struct card {
const char *face; // define pointer face
const char *suit; // define pointer suit
};
```



Programming Fundamentals Lab Manual



```
int main(void) {
    struct card myCard; // define one struct card variable
    // place strings into myCard
    myCard.face = "Ace";
    myCard.suit = "Spades";

    struct card *cardPtr = &myCard; // assign myCard

    printf("%s of %s\n", myCard.face, myCard.suit);
    printf("%s of %s\n", cardPtr->face, cardPtr->suit);
    printf("%s of %s\n", (*cardPtr).face, (*cardPtr).suit);
}
```

Output

```
Ace of Spades
Ace of Spades
Ace of Spades
```

Passing Structures to Functions

In C, structures can be passed to functions in three main ways:

- 1. Pass individual structure members**

A single member of a structure, such as an int or float, can be passed to a function. This is done by value, so the function cannot modify the original member in the caller.

- 2. Pass the entire structure object**

The entire structure can be passed to a function. This is also done by value, meaning the structure is copied. Any modifications inside the function do not affect the original structure.

- 3. Pass a pointer to the structure object**

The address of the structure (&structVar) can be passed to a function. This allows modification of the original structure and is more efficient than passing the whole structure by value, especially for large structures, because no copying is required.

Example

```
#include <stdio.h>

// Define a structure
struct Point {
    int x;
    int y;
};
```



Programming Fundamentals Lab Manual



```
// Function to pass individual members by value
void modifyMembers(int a, int b) {
    a = 100;
    b = 200;
}

// Function to pass entire structure by value
void modifyStructure(struct Point p) {
    p.x = 100;
    p.y = 200;
}

// Function to pass structure by reference using a pointer
void modifyStructurePointer(struct Point *p) {
    p->x = 100;
    p->y = 200;
}

int main() {
    struct Point pt = {10, 20};

    // Pass individual members
    modifyMembers(pt.x, pt.y);
    printf("After modifyMembers: x = %d, y = %d\n", pt.x, pt.y); // x=10,
y=20
    // Pass entire structure
    modifyStructure(pt);
    printf("After modifyStructure: x = %d, y = %d\n", pt.x, pt.y); //
x=10, y=20
    // Pass structure by reference
    modifyStructurePointer(&pt);
    printf("After modifyStructurePointer: x = %d, y = %d\n", pt.x,
pt.y); // x=100, y=200

    return 0;
}
```

Output

```
After modifyMembers: x = 10, y = 20
After modifyStructure: x = 10, y = 20
After modifyStructurePointer: x = 100, y = 200
```



Programming Fundamentals Lab Manual



Passing an Array Using Structure

Arrays of structure objects behave like all arrays in C: they are automatically passed by reference. This means the function can modify the elements of the array directly.

Example

```
#include <stdio.h>

struct ArrayWrapper {
    int arr[5];
};

void modify(struct ArrayWrapper aw) {
    aw.arr[0] = 100; // only modifies the copy
}

int main() {
    struct ArrayWrapper aw = {{1, 2, 3, 4, 5}};
    modify(aw);
    printf("%d\n", aw.arr[0]); // prints 1, original array not changed
}
```

typedef

The keyword typedef enables you to create synonyms (or aliases) for previously defined types. It's commonly used to create shorter names for struct types and simplify declarations of types like function pointers.

```
typedef existing_type NewTypeName;
```

Note: By convention, the first letter of a typedef name is capitalized to distinguish it from variable names.

Using typedef with Structures

```
struct card {
    const char *face;
    const char *suit;
};

typedef struct card Card;
```



Programming Fundamentals Lab Manual

Synonyms for Built-in Data Type

The typedef keyword is used to create aliases for built-in data types to improve code portability and readability. Since the size of types such as int and long may vary across different systems, a typedef allows a consistent type name to represent a required data size. When porting a program to another platform, only the typedef

definition needs modification, ensuring maintainable and portable code.

```
typedef float Real;
```

Example

```
#include <stdio.h>

/* Structure definition with typedef */
typedef struct student
{
    int roll_no;
    float percentage;
} Student;

int main(void)
{
    Student s1;

    s1.roll_no = 1;
    s1.percentage = 88.5;

    printf("Roll No: %d\n", s1.roll_no);
    printf("Percentage: %.2f\n", s1.percentage);

    return 0;
}
```

Output

```
Roll No: 1
Percentage: 88.50
```

Unions

A union is similar to a struct in that it can store members of different data types. However, there are some differences:

- In a struct, each member has its own memory.



Programming Fundamentals Lab Manual

- In a union, all members share the same memory, which means you can only use one of the values at a time. A union can contain members of different data types, but only one member can store a value at any given time. The amount of memory required to store a union is implementation-dependent. The memory allocated for a union is sufficient to hold its largest member. The sizeof () operator always returns a value greater than or equal to the size of the largest member.

Declaration of Union

A union definition creates a new data type but does not allocate memory until a variable of that type is declared. Union definitions are typically placed in header files for shared use. To create a union, use the union keyword, and then create a variable from it (just like with structs):

Case 1: Using a union tag name (recommended)

```
union union_tag {
    DataType Variable_1;    // Member
    DataType Variable_2;    // Member
    . . .
    DataType Variable_n;    // Member
};
union union_tag union_variable1;
union union_tag union_variable2;
```

Case 2: Without using a tag name (tag name optional)

```
union union_tag {          //No tag
    DataType Variable_1;    // Member
    DataType Variable_2;    // Member
    . . .
    DataType Variable_n;    // Member
} var1, var2;              // Must declare variables immediately
```

Allowed Union Operations

The following operations are permitted on union variables:

- Assigning a union to another union of the same type
- Obtaining the address of a union variable using the address-of operator (&)
- Accessing union members using the structure member operator (.) and the structure pointer operator (->)
- Zero initialization of a union

Example

```
#include <stdio.h>
union number /* Union definition */
{
    int x;
    double y;
};

int main(void)
{
    union number u1 = {0}; /* Zero initialization */
```



Programming Fundamentals Lab Manual



```
/* Accessing members using dot operator */
u1.x = 25;
printf("u1.x = %d\n", u1.x);

/* Assigning one union to another of the same type */
union number u2;
u2 = u1;
printf("u2.x (after assignment) = %d\n", u2.x);

/* Taking address of a union variable */
printf("Address of u1 = %p\n", (void *)&u1);

/* Using sizeof with union */
printf("Size of union number = %zu bytes\n", sizeof(union number));

return 0;
}
```

Output

```
u1.x = 25
u2.x (after assignment) = 25
Address of u1 = 00000077417FFB68
Size of union number = 8 bytes
```

Note: Comparison of two unions using the equality (==) and inequality (!=) operators is not allowed, for the same reasons that structure variables cannot be directly compared.

Initialization of Union

A union can be initialized at the time of declaration using a value of its first member type.

```
union union_tag {
    DataType Variable_1;    // Member
    DataType Variable_2;    // Member
    . . .
    DataType Variable_n;    // Member
};

/* Correct initialization with first member type */
union union_tag variable1 = {value_of_member1};
/* Initialization with different type (may cause truncation or warning) */
union union_tag variable2 = {value_of_different_type};
/* Assigning to another member after declaration */
variable2.member2 = value_of_member2;
```



Programming Fundamentals Lab Manual



Example

```
#include <stdio.h>
union number
{
    int x;        // first member
    double y;     // second member
};

int main(void)
{
    /* Initialization with first member type */
    union number value1 = {10};
    printf("value1.x = %d\n", value1.x);

    /* Initialization with different type (double), truncated to int */
    union number value2 = {1.43};
    printf("value2.x (truncated) = %d\n", value2.x); // Prints the first
    member x; 1.43 is truncated to 1
    // Accessing inactive member y; prints unpredictable value (memory of
    x interpreted as int)
    printf("value2.y (truncated) = %d\n", value2.y);

    /* Assigning to second member after declaration */
    value2.y = 1.43;
    printf("value2.y = %.2f\n", value2.y);

    return 0;
}
```

Output

```
value1.x = 10
value2.x (truncated) = 1
value2.y (truncated) = 1
value2.y = 1.43
```



Programming Fundamentals Lab Manual



Note: Only the first member of a union can be directly initialized in a declaration; initializing with a type different from the first member may cause data loss or compiler warnings, because unions store only one member at a time and the initializer always applies to the first member.

Anonymous Structures and Unions

Anonymous structs and unions can be nested inside named structs or unions. Members of a nested anonymous struct or union become direct members of the enclosing struct or union and can be accessed directly through an object of the outer type, without specifying a nested name.

A struct is a way to group related data together. Sometimes, a struct can contain another struct or union inside it. This is called a **nested struct/union**.

```
struct Outer {
    int a;
    int b;
    struct Inner {    // named nested struct
        int x;
        int y;
    } inner;

    union Data {      // named nested union
        int i;
        float f;
    } data;
};

// Access members
struct Outer obj;
obj.a = 10;
obj.inner.x = 5;    // access nested struct member
obj.data.i = 100;   // access nested union member
```

For anonymous structs and unions

```
struct Example {
    int a;
    int b;
    struct {    // anonymous struct
        int x;
        int y;
    };        // no name
    union {     // anonymous union
        int i;
        float f;
    };        // no name
};
```



Programming Fundamentals Lab Manual



```
// Access members
struct Example obj;
obj.a = 10;
obj.x = 5;    // direct access, no struct name needed
obj.i = 100;  // direct access, no union name needed
```

Example

```
#include <stdio.h>

// Outer struct
struct DataContainer {
    int id; // regular member

    // Named nested struct
    struct NamedStruct {
        int x;
        int y;
    } named;

    // Anonymous nested struct
    struct {
        int a;
        int b;
    }; // members promoted to outer struct

    // Named nested union
    union NamedUnion {
        int i;
        float f;
    } namedUnion;

    // Anonymous nested union
    union {
        int m;
        float n;
    }; // members promoted to outer struct
};
```



Programming Fundamentals Lab Manual



```
int main() {
    struct DataContainer obj;

    // Access outer struct members
    obj.id = 1;

    // Access named struct members
    obj.named.x = 10;
    obj.named.y = 20;

    // Access anonymous struct members directly
    obj.a = 30;
    obj.b = 40;

    // Access named union members
    obj.namedUnion.i = 100;

    // Access anonymous union members directly
    obj.m = 200;

    // Print all values
    printf("id = %d\n", obj.id);
    printf("named.x = %d, named.y = %d\n", obj.named.x, obj.named.y);
    printf("a = %d, b = %d\n", obj.a, obj.b);
    printf("namedUnion.i = %d\n", obj.namedUnion.i);
    printf("m = %d\n", obj.m);

    return 0;
}
```

Output

```
id = 1
named.x = 10, named.y = 20
a = 30, b = 40
namedUnion.i = 100
m = 200
```



Programming Fundamentals Lab Manual



Lab Task

Task 1: Define a structure named Student that contains three members: an integer id, a character array name of size 30, and a floating-point for marks. Declare a variable of this structure type, initialize it with suitable values, and display all its members using printf in a readable format. Optionally, try initializing the structure using both sequential and designated initializers to observe how the values are assigned.

C Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Student {
5     int id;
6     char name[30];
7     float marks;
8 };
9
10 int main() {
11     // trying the normal sequential way first
12     struct Student s1 = {16, "Umar", 95.5};
13
14     // doing designated initialization for the second one
15     struct Student s2 = {
16         .marks = 92.0,
17         .id = 32,
18         .name = "Haseeb"
19     };
20
21     printf("student 1 -> id: %d, name: %s, marks: %.1f\n", s1.id, s1.name, s1.marks);
22     printf("student 2 -> id: %d, name: %s, marks: %.1f\n", s2.id, s2.name, s2.marks);
23
24     return 0;
25 }
```

Output

```
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_016$ make task1
gcc -c task1.c
gcc -o task1 task1.o
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_016$ ./task1
student 1 -> id: 16, name: Umar, marks: 95.5
student 2 -> id: 32, name: Haseeb, marks: 92.0
```



Programming Fundamentals Lab Manual



Task 2: Define a union named Numeric that contains three members: an integer intValue, a floating-point floatValue, and a nested structure Parts which itself has two short members, high and low. Declare a variable of this union type, assign values to each of its members carefully, and display the values to observe how the union stores and shares memory among its members.

C Code

```
#include <stdio.h>

union Numeric {
    int intValue;
    float floatValue;
    struct {
        short low;
        short high;
    } Parts;
};

int main() {
    union Numeric num;

    // setting the int value first
    num.intValue = 42;
    printf("after setting int:\n");
    printf("intValue: %d\n", num.intValue);
    printf("floatValue (garbage): %f\n\n", num.floatValue);
```

```
1 // now setting the float, which overwrites the int memory entirely
2 num.floatValue = 3.14f;
3 printf("after setting float:\n");
4 printf("intValue (garbage): %d\n", num.intValue);
5 printf("floatValue: %f\n\n", num.floatValue);
6
7 // messing with the nested struct
8 num.Parts.low = 10;
9 num.Parts.high = 20;
10 printf("after setting parts:\n");
11 printf("low: %d, high: %d\n", num.Parts.low, num.Parts.high);
12 // intValue will be something completely different now because of memory sharing
13 printf("intValue (overwritten): %d\n", num.intValue);
14
15 return 0;
16 }
```



Programming Fundamentals Lab Manual



Output

```
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_01a$ ./task2
after setting int:
intValue: 42
floatValue (garbage): 0.000000

after setting float:
intValue (garbage): 1078523331
floatValue: 3.140000

after setting parts:
low: 10, high: 20
intValue (overwritten): 1310730
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_01a$
```

Task 3: Corrected the errors in the Given Code and show the output.

Bugged Code

```
#include <stdio.h>
struct Sensor
{
    int id;
    char name[20];

    union
    {
        int intValue;
        float floatValue;
        struct
        {
            unsigned int status : 3;
            unsigned int mode : 2;
        };
    };
};

int main()
{
    struct Sensor s;
    s.id = "001";
    s.name = 'TempSensor';
    s.intValue = 100.5;
    s.floatValue = 25;
    s.status = 5;
    s.mode = 3;

    if (s.status = 4 && s.mode > 2)
        printf("Sensor is active\n");
    else
        printf("Sensor is inactive\n");
    printf("Sensor ID: %d\n", s.ID);
    printf("Sensor Name: %s\n", s.name);
    printf("Int Value: %d\n", s.intValue);
    printf("Float Value: %f\n", s.floatValue);
    printf("Status: %u Mode: %u\n", s.status, s.mode);
}
```



Programming Fundamentals Lab Manual



Corrected Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Sensor {
5     int id;
6     char name[20];
7     union {
8         int intValue;
9         float floatValue;
10        struct {
11            unsigned int status : 3;
12            unsigned int mode : 2;
13        };
14    };
15 };
16
17 int main() {
18     struct Sensor s;
19
20     // fixed: id is just an int, not a string
21     s.id = 1;
22
23     // fixed: you have to use strcpy for strings in c
24     strcpy(s.name, "TempSensor");
25
26     // union sharing time. setting float last so it's the active one in memory
27     s.intValue = 100;
28     s.floatValue = 25.0;
29
30
31     // bitfields
32     s.status = 5;
33     s.mode = 3;
34
35     // fixed: was using = instead of ==
36     if (s.status == 4 && s.mode > 2) {
37         printf("Sensor is active\n");
38     } else {
39         printf("Sensor is inactive\n");
40     }
41
42     // fixed: case sensitivity on s.id
43     printf("Sensor ID: %d\n", s.id);
44     printf("Sensor Name: %s\n", s.name);
45
46     // intValue will look like nonsense here because the float is occupying the memory
47     printf("Int Value (shared): %d\n", s.intValue);
48     printf("Float Value: %f\n", s.floatValue);
49     printf("Status: %u, Mode: %u\n", s.status, s.mode);
50
51     // fixed: main needs to return an int
52     return 0;
53 }
```



Programming Fundamentals Lab Manual



Output

```
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_016$ ./task3
Sensor is inactive
Sensor ID: 1
Sensor Name: TempSensor
Int Value (shared): 1103626269
Float Value: 25.000055
Status: 5, Mode: 3
```

Task 4: Define a struct named Employee that contains the employee id (integer), name (string of size 30), and an anonymous union. The union should include two members: a float salary and an anonymous struct containing two integers: hoursWorked and overtimeHours. Declare a variable of this Employee structure, assign values to all members of the union using direct assignment, and then display the values. Demonstrate how assigning a value to one member of the union affects the others, showing how memory is shared within the union.

C Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 struct Employee {
5     int id;
6     char name[30];
7     // anonymous union
8     union {
9         float salary;
10        // anonymous struct inside the union
11        struct {
12            int hoursWorked;
13            int overtimeHours;
14        };
15    };
16 };
17
18 int main() {
19     struct Employee emp;
20
21     emp.id = 777;
22     strcpy(emp.name, "hassan");
23
24     // let's assign the nested struct values directly
25     emp.hoursWorked = 40;
26     emp.overtimeHours = 5;
27
28     printf("before touching salary:\n");
29     printf("hours: %d, overtime: %d\n", emp.hoursWorked, emp.overtimeHours);
30     printf("salary (garbage): %f\n", emp.salary);
31 }
```

```
// now we assign salary, which will blast away the hours data since they share memory
emp.salary = 50000.50f;

printf("after setting salary:\n");
printf("salary: %.2f\n", emp.salary);
printf("hours (overwritten): %d\n", emp.hoursWorked);
printf("overtime (overwritten): %d\n", emp.overtimeHours);

return 0;
```



Programming Fundamentals Lab Manual



Output

```
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_016$ make task4
gcc -c task4.c
gcc -o task4 task4.o
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_016$ ./task4
before touching salary:
hours: 40, overtime: 5
salary (garbage): 0.000000

after setting salary:
salary: 50000.50
hours (overwritten): 1195593856
overtime (overwritten): 5
umar@umarr:~/Downloads/PF-Labs-main/Lab13_2025_EE_016$
```